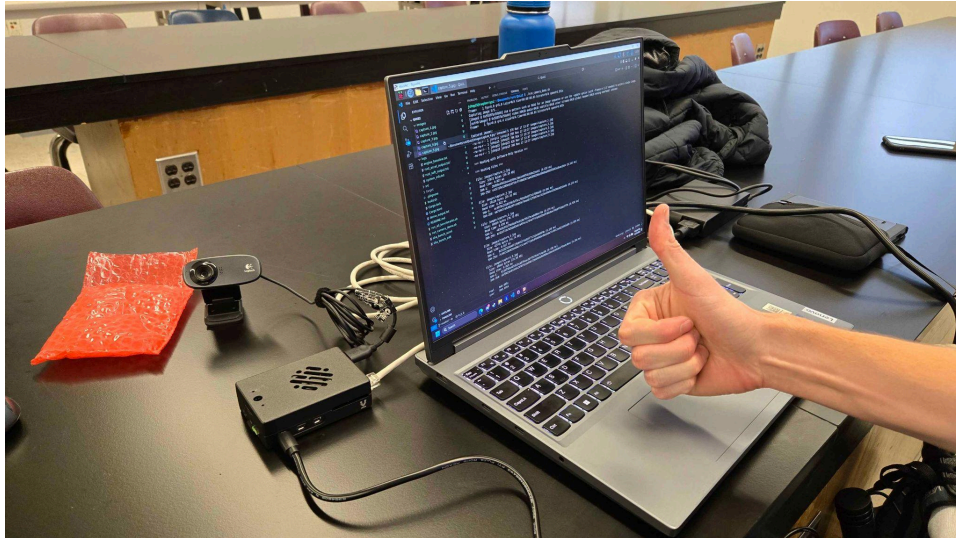


Quiz 2: SHA-1 and SHA-2 on Raspberry Pi

Garima Bhattarai | Mia Diliberti-Brandenburg | Marcelo P. Masilungan
John Pollak | Kelly Williams

Setup and Method

The hardware setup for this experiment was fairly simple, just a Raspberry Pi 5 with a USB camera connected to a laptop via Ethernet.



To run our software, we need to create a project directory and clone our GitHub repository. Besides any dependencies included inside Cargo.toml, we need to install the Rustup toolchain and a camera tool: we used a USB webcam, so we installed the package “`rpikam-apps`”. The benchmark suite has a few different tools: a benchmark tool for software, hardware extensions, openssl (baseline), and camera.

We ran ten trials per digest size to get a more robust set of data. During every SHA benchmark, with the exception of the camera benchmark, a randomized set of data of a specific size is hashed. An average is taken of the ten trials performed, and the results are output to a table. Data is output in milliseconds and throughput (MB/s) for the Rust benchmarks, and seconds and throughput (kB/s) for the engine baseline in OpenSSL. The logs for the “standard” benchmark script are in the logs file, and the camera testing logs are written to `demo_output.txt` in the main folder; pictures taken during the benchmark are inside the “images” folder. During the camera benchmark, five photos are taken and hashed with SHA-1 and SHA-256 in Rust, first with software only, and second with hardware acceleration.

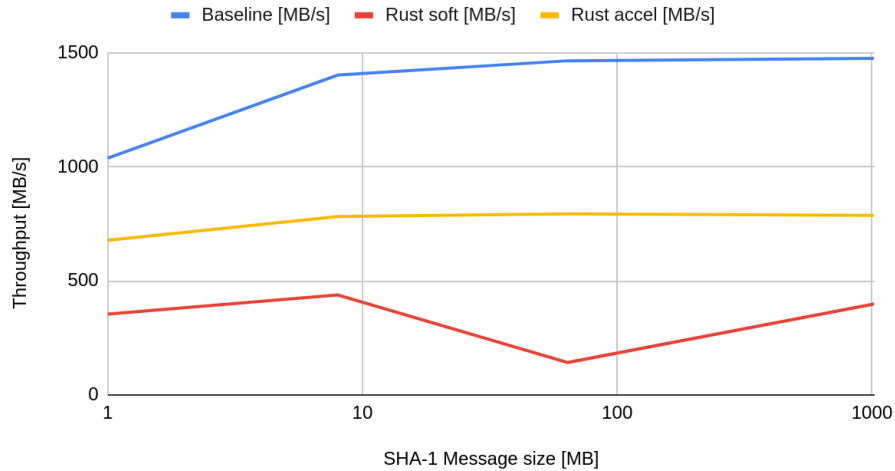
Data for the standard benchmark will be shown in plots on page 2, and plots for the camera will be on page 2.

Quiz 2

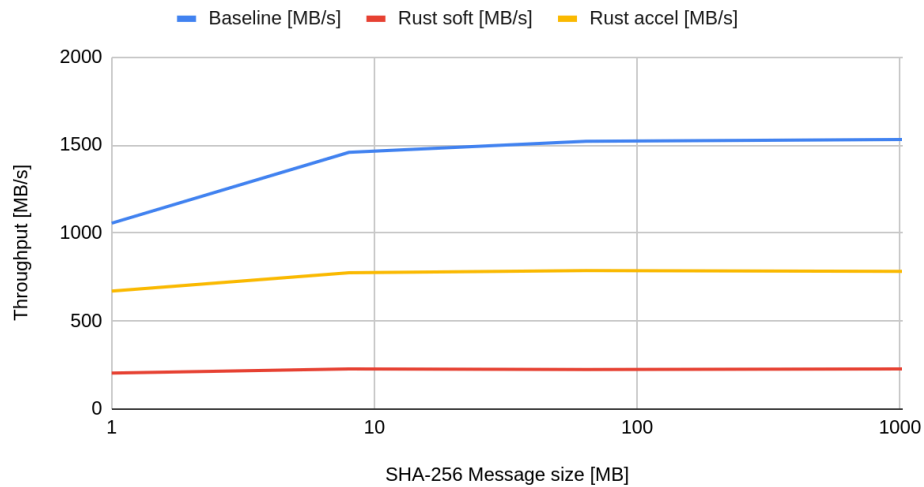
Group C

Plots and Tables

SHA-1 Throughput vs Message Size



SHA-256 Throughput vs Message Size



SHA-1 (160)	img 1 hw	img 1 sw	img 2 hw	img 2 sw	img 3 hw	img 3 sw	img 4 hw	img 4 sw	img 5 hw	img 5 sw	sw avg	hw avg
Time [ms]	0.027	0.132	0.019	0.091	0.066	0.278	0.031	0.148	0.022	0.106	0.151	0.0275

SHA-256	img 1 hw	img 1 sw	img 2 hw	img 2 sw	img 3 hw	img 3 sw	img 4 hw	img 4 sw	img 5 hw	img 5 sw	sw avg	hw avg
Time [ms]	0.029	0.343	0.02	0.174	0.061	0.536	0.033	0.285	0.023	0.204	0.3084	0.02766

Analysis

ARMv8 SHA instructions, which are on by default, seem to have a significant impact on the speed of hashing regardless of what is used. The OpenSSL library with SHA extensions is fastest, which makes sense, because it is probably the best optimized version of both SHA-1 and SHA-256, compared to our Rust implementation. However, the speedup from the software-based to extension-accelerated Rust implementation is more than three times higher throughput for SHA-256 and about double or slightly more than double throughput for SHA-1, increasing with message size, and significantly faster. In the camera benchmarks, the difference is even more stark: hardware accelerated speeds are more than ten times faster to finish in SHA-256, and a bit more than five times faster in SHA-1.

For smaller packets, OpenSSL will always be the best choice; however, when choosing between the Rust implementations, the hardware accelerated version will always run faster. OpenSSL is highly optimized for small packets. Hardware-accelerated Rust implementations benefit from having hardware hash instructions, which increases throughput and reduces CPU load. In any case, SHA-1 will have slightly higher throughput and speed. However, the speed tradeoff is small enough that SHA-256 should be preferred over SHA-1, which is deprecated for security reasons.¹

The speed difference between the hardware accelerated and software versions for larger sizes in SHA-256 means that, ideally, the software version should *never* be used for larger buffer sizes. The speed difference between SHA-256 and SHA-1 is negligible at higher message sizes with acceleration, so SHA-256 should be used unless SHA-1 would not be a security problem or there are other issues that would keep it in place.

The slightly lower speed and throughput from SHA-256 is due to not only a longer digest size (256 bits for SHA-2 vs 160 bits for SHA-1) but also to increased complexity: both functions use Merkle–Damgård construction, but SHA-2 adds logic from Davies–Meyer compression function.² This additional logic and larger digest size reduces the number of collisions so that the function is more secure, but results in higher computational complexity.

Our results show that the Raspberry Pi 5's ARMv8 SHA extensions provide consistent performance improvements across all tested configurations, demonstrating the strong impact of hardware support on hashing throughput. These findings highlight how buffer size, implementation strategy, and available hardware features collectively influence overall hashing performance on embedded systems.

¹ <https://csrc.nist.gov/projects/hash-functions>

² <https://www.geeksforgeeks.org/computer-networks/difference-between-sha1-and-sha2/>